Instituto Tecnológico Superior de Xalapa

Instituto Tecnológico Superior de Xalapa

Estructura de Datos

Tema: Tarea 1 unidad 3 Pilas estáticas y dinámicas en Java

Alumno: Mauricio Ahued Cortés

Matrícula: 257O02229

Maestro: Víctor Hugo Vásquez Herrera

Xalapa, Veracruz, México
11 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Índice

Introducción y comportamiento LIFO 3
Pila estática con arreglo 4
Pila dinámica con nodos 5
Pruebas de funcionamiento 6
Evidencia de la ejecución 7
Rendimiento y uso de memoria 8

Xalapa, Veracruz, México
11 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Introducción y comportamiento LIFO

Una pila inserta y retira elementos por un mismo extremo, llamado tope o cima. Sigue la regla LIFO (Last In, First Out): el último dato que entra es el primero que sale.

La práctica compara dos formas de almacenar enteros con el mismo comportamiento. PilaSimple utiliza un arreglo de cinco posiciones y un índice llamado tope. PilaDinamica utiliza nodos que contienen un dato y una referencia al nodo de abajo; cima identifica el primer nodo.

push(int x) agrega un elemento, pop() retira el último y mostrar() imprime la pila. Las consultas isEmpty() y peek() permiten conocer su estado sin retirar datos. La implementación estática también utiliza isFull() para verificar su capacidad fija.

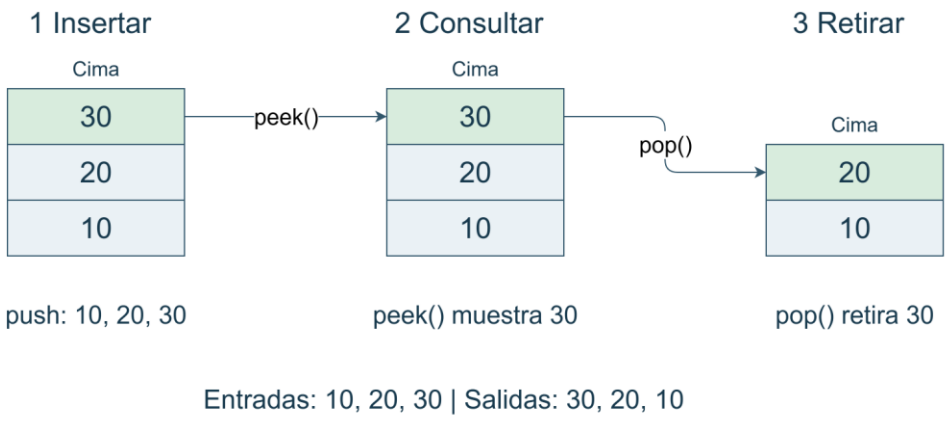


Figura 1 Inserción consulta y extracción de una pila

Al insertar 10, 20 y 30, el valor 30 queda arriba. peek() muestra ese valor sin retirarlo. El siguiente pop() saca el 30; después se retiran 20 y 10. El orden es el mismo en ambas pilas, aunque los datos se almacenen de distinta forma.

Cuando una pila está vacía no puede retirarse ningún elemento. En la pila estática también se rechaza una inserción cuando el arreglo está lleno. Estos controles evitan acceder a posiciones o nodos inexistentes.

Xalapa, Veracruz, México
11 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Pila estática con arreglo

El arreglo conserva la capacidad de cinco enteros del ejemplo de clase. tope comienza en -1; al insertar aumenta una posición y al retirar disminuye una posición. Los elementos permanecen en las posiciones consecutivas del arreglo.

Inserción y extracción

```
public void push(int x) {
    if (!isFull()) {
        tope++;
        datos[tope] = x;
        System.out.println("Metiste: " + x);
    } else {
        System.out.println("Pila llena: no se pudo insertar " + x + ".");
    }
}

public void pop() {
    if (!isEmpty()) {
        System.out.println("Sacaste: " + datos[tope]);
        tope--;
    } else {
        System.out.println("Pila vacia: no hay elementos para sacar.");
    }
}
```

Consultas del estado

```
public boolean isEmpty() {
    return tope == -1;
}

public boolean isFull() {
    return tope == datos.length - 1;
}

// Consulta sin cambiar el tope.
public void peek() {
    if (isEmpty()) {
        System.out.println("Pila vacia: no hay un dato para consultar.");
    } else {
        System.out.println("Dato en el tope: " + datos[tope]);
    }
}
```

isEmpty() comprueba si tope vale -1 e isFull() comprueba si llegó al último índice del arreglo. peek() muestra datos[tope] cuando hay elementos; si está vacía, muestra un mensaje. La consulta no cambia el índice.

mostrar() conserva el recorrido del código de clase: imprime desde el fondo hasta el tope. La dirección se indica en la consola, por lo que el último número impreso es el próximo que retira pop().

Xalapa, Veracruz, México
11 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Pila dinámica con nodos

Cada nodo guarda un entero y una referencia al siguiente nodo. cima inicia en null. push() enlaza el nodo nuevo con la cima anterior; pop() mueve cima al nodo que queda debajo.

Nodo e inserción

```
int dato;
Nodo siguiente; // Referencia al nodo que queda debajo.

public Nodo(int x) {
    dato = x;
    siguiente = null;
}

public void push(int x) {
    Nodo nuevo = new Nodo(x);
    nuevo.siguiente = cima; // Conserva el enlace con la cima anterior.
    cima = nuevo;
}
```

Extracción y consultas

```
public void pop() {
    if (!isEmpty()) {
        System.out.println("Sacaste de la pila: " + cima.dato);
        cima = cima.siguiente;
    } else {
        System.out.println("Pila dinamica vacia: no hay elementos para sacar.");
    }
}

public boolean isEmpty() {
    return cima == null;
}

// Consulta sin quitar el nodo.
public void peek() {
    if (isEmpty()) {
        System.out.println("Pila vacia: no hay un dato para consultar.");
    } else {
        System.out.println("Dato en la cima: " + cima.dato);
    }
}
```

isEmpty() comprueba si cima es null. peek() muestra el dato de la cima sin quitar el nodo. mostrar() recorre los enlaces desde la cima hasta encontrar null.

La pila dinámica no necesita isFull() porque no tiene un número fijo de posiciones. Puede seguir creando nodos mientras haya memoria disponible.

Instituto Tecnológico Superior de Xalapa

Pruebas de funcionamiento

Se compilaron los cinco fuentes y se ejecutaron ambos lanzadores. Las salidas completas se guardaron como texto en el README. Se comprobó el orden de extracción, la consulta con peek() y los controles de pila vacía y llena.

Caso	Pila estática	Pila dinámica
Estado inicial	isEmpty(): true.	isEmpty(): true.
Insertar 10 20 30	peek muestra 30; mostrar imprime 10 20 30.	peek muestra 30; mostrar imprime 30 20 10.
Consultar con peek	No cambia tope ni retira el 30.	No cambia cima ni quita el nodo con 30.
Retirar el dato de arriba	pop retira 30; quedan 10 y 20.	pop retira 30; quedan 20 y 10.
Sexto elemento	Rechaza 60; quedan cinco datos. Valor del tope: 50.	Acepta 60; quedan seis nodos. Valor de la cima: 60.
Vaciar y retirar otra vez	isEmpty(): true; peek y pop muestran pila vacía.	isEmpty(): true; peek y pop muestran pila vacía.

Resultados de los límites

El intento de desbordamiento dejó intacta la pila estática. La salida real fue:

```
Esta llena: true
Pila llena: no se pudo insertar 60.
Pila actual (Fondo -> Tope): 10 20 30 40 50
Dato en el tope: 50
```

La pila dinámica conservó seis nodos y retiró primero el último insertado:

```
Pila dinamica (Cima -> Fondo): 60 50 40 30 20 10
Dato en la cima: 60
```

Xalapa, Veracruz, México
11 de octubre de 2026

Instituto Tecnológico Superior de Xalapa

Evidencia de la ejecución

La captura muestra la ejecución de la pila estática en Visual Studio Code: la consulta con peek(), el intento de insertar un sexto dato y la extracción de los elementos hasta vaciar la pila.

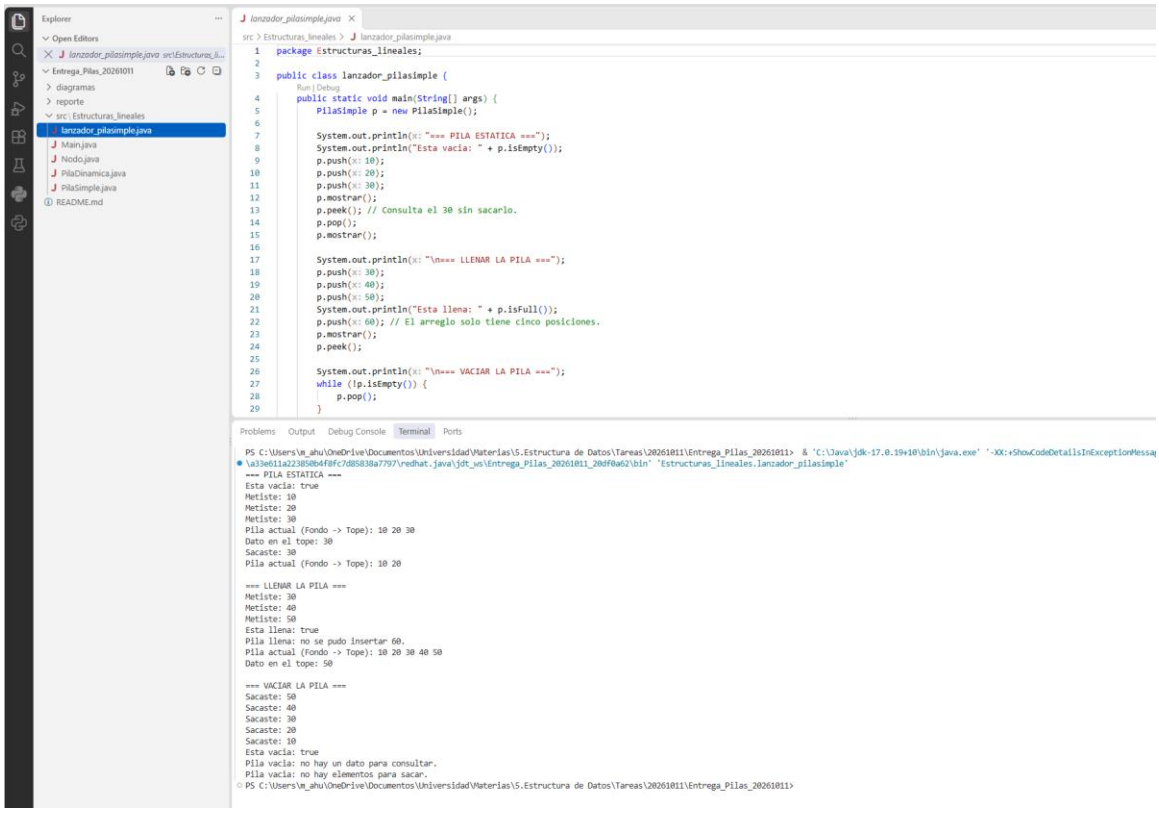


Figura 2. Ejecución de la pila estática.

La consulta conserva el 30 en el tope. Al llenar las cinco posiciones, el programa rechaza el 60. Después retira 50, 40, 30, 20 y 10; al quedar vacía, muestra los mensajes de consulta y extracción sin elementos.

Xalapa, Veracruz, México
11 de octubre de 2026

Rendimiento y uso de memoria

En ambas pilas se inserta, retira y consulta directamente en el extremo de arriba, sin recorrer todos los datos. Para mostrar el contenido completo sí se necesita recorrer la estructura.

Aspecto	Pila estática	Pila dinámica
push y pop	Actualizan el índice y una posición del arreglo.	Actualizan los enlaces; push crea un nodo.
peek e isEmpty	Consultan el índice o el dato del tope.	Consultan la referencia cima o su dato.
isFull	Compara tope con el último índice del arreglo.	No se utiliza.
mostrar	Recorre los datos guardados en el arreglo.	Recorre los nodos enlazados.
Memoria	Reserva el arreglo completo de cinco posiciones.	Crea un nodo por dato y guarda sus enlaces.
Capacidad	Fija: cinco datos en esta práctica.	Variable según la memoria disponible.

La comparación de rendimiento y memoria es teórica. Las ejecuciones muestran el funcionamiento de las pilas, pero no se midieron tiempos ni consumo de memoria.

Conclusiones

Considero que la pila estática es más sencilla cuando se conoce cuántos datos se van a guardar. El arreglo reserva todas sus posiciones desde el inicio, aunque algunas queden vacías. Si se llena, ya no puede recibir otro dato.

La dinámica permite agregar nodos conforme se necesitan. Cada nodo también guarda una referencia al siguiente, por lo que requiere memoria adicional para los enlaces y los objetos.

Usaría la estática si el límite de datos es conocido y la dinámica si la cantidad cambia. Las dos permiten trabajar directamente con el dato de arriba. Con estas ejecuciones no puedo afirmar que una sea más rápida; la diferencia que se observa es la capacidad fija del arreglo frente al crecimiento por nodos.

Xalapa, Veracruz, México
11 de octubre de 2026